

PREPARACION PARA LA PRACTICA EXPERIMENTAL UNIDAD I – PASO 3

CSS3: DE CERO A LA PRACTICA

Teoria y ejemplos progresivos para el Paso 3 de la PE U1

Reset → Unidades → Variables → Box Model → Flexbox → Grid → Responsive → Modo Oscuro → Animaciones

Aplicaciones Web – Ingenieria de Software – UTEQ 2025-2026

Como usar este documento

Los ejemplos estan en el mismo orden que el CSS del Paso 3 de la practica. Cada concepto se explica con teoria minima y un ejemplo que se puede probar creando un archivo `.html` con un `<style>` interno. Al final, el estudiante habra entendido cada linea del CSS que debe escribir en la practica.

Índice

1. Nivel 1: Como funciona CSS y el reset minimo	2
1.1. Que es CSS y como se conecta al HTML	2
1.2. El selector, la propiedad y el valor	2
1.3. El reset: por que es obligatorio	2
2. Nivel 2: Unidades de medida	3
3. Nivel 3: Variables CSS (Custom Properties)	4
4. Nivel 4: Box Model – margin, padding, border	6
5. Nivel 5: Flexbox – distribuir en una dimension	7
6. Nivel 6: CSS Grid – layout de pagina completo	9
7. Nivel 7: Responsive mobile-first con media queries	10
8. Nivel 8: Estilos de formulario accesibles	11
9. Nivel 9: Botones con transiciones y estados	12
10.Nivel 10: Modo oscuro automatico y manual	14
11.Nivel 11: Animaciones y accesibilidad	16
11.1. Enlace de salto (skip link) accesible	17
12.Nivel FINAL: Mapa completo del CSS de la practica	17

Nivel 1: Como funciona CSS y el reset minimo

1.1 Que es CSS y como se conecta al HTML

Concepto: CSS

CSS (Cascading Style Sheets) define la **apariencia visual** del HTML: colores, tamanos, posiciones, espaciado, animaciones. Se conecta al HTML de 3 formas: (1) archivo externo con `<link>` (la mejor), (2) etiqueta `<style>` en el `<head>`, (3) atributo `style=""` inline (la peor, no usar).

Ejemplo 1 – Conectar CSS externo al HTML

```
1  <!-- En el <head> del HTML: -->
2  <link rel="stylesheet" href="css/main.css">
```

1.2 El selector, la propiedad y el valor

Ejemplo 2 – Anatomia de una regla CSS

```
1  /* selector { propiedad: valor; } */
2
3  h1 {
4      color: #0066cc;          /* color del texto */
5      font-size: 2rem;         /* tamaño de la fuente */
6      margin-bottom: 1rem;     /* espacio debajo */
7  }
8
9  /* Selector de clase: aplica a todo elemento con class="tarjeta" */
10 .tarjeta {
11     background: #f8f9fa;
12     padding: 1.5rem;
13 }
14
15 /* Selector de ID: aplica a UN solo elemento con id="header" */
16 #header {
17     background: #0066cc;
18 }
```

1.3 El reset: por que es obligatorio

Concepto: Reset CSS

Cada navegador tiene estilos por defecto diferentes. Chrome pone un margen de 8px al body, Firefox pone 8px pero con reglas distintas para listas y tablas, Safari tiene otra variante. Estas diferencias causan que un sitio se vea diferente en cada navegador.

El reset elimina esas diferencias estableciendo una base comun. Existen dos enfoques principales: (1) el **reset completo** (como el de Eric Meyer) que elimina absolutamente todos los estilos por defecto, y (2) el **reset minimo** (como Normalize.css) que solo corrige las inconsistencias. En la practica usamos un reset minimo de 3 reglas porque es suficiente para proyectos modernos.

El selector `*` (asterisco) selecciona todos los elementos del documento. Los pseudo-elementos `::before` y `::after` se incluyen porque tambien son cajas que necesitan el mismo `box-sizing`.

Ejemplo 3 – Reset minimo (el que usa la practica)

```
1  /* Aplicar a TODOS los elementos */
2  *, ::before, ::after {
3      box-sizing: border-box; /* padding y border incluidos en width */
4      margin: 0;             /* quitar margenes por defecto */
5      padding: 0;            /* quitar padding por defecto */
6  }
7
8  /* Imagenes y videos: nunca desbordar su contenedor */
9  img, video {
10     max-width: 100%;
11     display: block;
12 }
```

Concepto: `box-sizing: border-box`

Sin `border-box`: un elemento con `width: 200px` y `padding: 20px` mide 240px ($200 + 20 + 20$). Con `border-box`: mide 200px (el padding se resta del width). Sin esto, los layouts se rompen constantemente.

Donde se usa en la practica: `main.css` – primeras lineas

Estas 3 reglas son las primeras lineas del archivo `main.css` de la practica. Sin ellas, todo el layout posterior se comporta de forma impredecible.

Nivel 2: Unidades de medida

Concepto: `px` vs `rem` vs `em` vs `%` vs `vw`

CSS tiene unidades absolutas y relativas. Entender la diferencia es fundamental para crear sitios que se adapten a las preferencias del usuario:

- **px** (pixel): unidad absoluta. $1\text{px} = 1$ punto en la pantalla. NO escala si el usuario cambia el tamaño de fuente en su navegador. Usar solo para bordes (`border: 1px`).
- **rem** (root em): relativa al `font-size` del elemento `<html>` (por defecto 16px). $1\text{rem} = 16\text{px}$, $1.5\text{rem} = 24\text{px}$. SI escala con la preferencia del usuario. Es la unidad recomendada para tipografía, padding y margin.
- **em**: relativa al `font-size` del elemento padre (no del root). Si un `<div>` tiene `font-size: 20px`, entonces $1\text{em} = 20\text{px}$ dentro de ese div. Es útil para componentes que deben escalar proporcionalmente, pero puede causar confusión cuando se anidan (“efecto cascada”).
- **%**: relativa al elemento padre. `width: 80%` = 80% del ancho del padre. Muy usada para layouts fluidos.

- **vw / vh:** relativa al viewport (ventana del navegador). 1vw = 1 % del ancho de la ventana, 1vh = 1 % del alto. Usada en tipografia fluida con `clamp()`.

Regla practica: usar **rem** para texto y espaciado, % para anchos de contenedores, **px** solo para bordes, y **vw** dentro de `clamp()` para tipografia fluida.

Ejemplo 4 – Comparacion de unidades

```
1  /* px: absoluto. 1px = 1 pixel. NO escala si el usuario
2  cambia el tamaño de fuente en su navegador. */
3  h1 { font-size: 32px; }
4
5  /* rem: relativo al font-size del <html> (por defecto 16px).
6  1rem = 16px. SI escala con la preferencia del usuario.
7  SIEMPRE preferir rem sobre px para texto y espaciado. */
8  h1 { font-size: 2rem; } /* 2 * 16px = 32px */
9
10 /* %: relativo al elemento padre.
11 width: 80% = 80% del ancho del padre. */
12 .contenedor { width: 80%; }
13
14 /* vw: relativo al ancho del viewport (ventana).
15 1vw = 1% del ancho de la ventana.
16 Util en tipografia fluida con clamp(). */
17 h1 { font-size: 4vw; } /* 4% del ancho de la ventana */
18
19 /* clamp(minimo, preferido, maximo):
20 Nunca menor de 1.75rem, idealmente 4vw,
21 nunca mayor de 2.5rem. */
22 h1 { font-size: clamp(1.75rem, 4vw, 2.5rem); }
```

Donde se usa en la practica: variables.css

Todas las variables de tipografia en la practica usan `clamp()` para que el texto sea fluido sin media queries: `-tamano-base: clamp(1rem, 1.5vw + 0.5rem, 1.125rem);`

Nivel 3: Variables CSS (Custom Properties)

Concepto: Variables CSS (Custom Properties)

Las variables CSS se declaran en `:root` con `-nombre: valor;` y se usan con `var(-nombre)`. Permiten cambiar toda la paleta de colores del sitio desde un solo lugar. Son la base del modo oscuro.

Por que usar variables en lugar de valores directos:

- **Mantenibilidad:** si el cliente pide cambiar el azul corporativo, se modifica UNA linea en `:root` en lugar de buscar y reemplazar en 50 lugares.
- **Consistencia:** todos los componentes usan exactamente el mismo azul, el mismo espaciado, el mismo radio de borde. No hay variaciones accidentales.

- **Modo oscuro:** al cambiar las variables dentro de una media query o un selector como `[data-tema=."oscuro"]`, TODOS los elementos que usan esas variables cambian automaticamente sin escribir reglas adicionales.
- **Lectura:** `color: var(--color-primario)` es mas descriptivo que `color: #0066cc`. El nombre de la variable explica su proposito.

El selector `:root` equivale al elemento `<html>` y garantiza que las variables esten disponibles en todo el documento. Se recomienda organizar las variables por categoria: colores, tipografia, espaciado, layout y transiciones.

Ejemplo 5 – Declarar y usar variables

```
1  /* Declarar en :root (accesibles en todo el documento) */
2  :root {
3      --color-primario: #0066cc;
4      --color-fondo:    #ffffff;
5      --color-texto:    #1a1a1a;
6      --esp-md:         1rem;
7      --radio-borde:    6px;
8  }
9  /* Usar con var() */
10 body {
11     background-color: var(--color-fondo);
12     color: var(--color-texto);
13 }
14
15 .btn {
16     background: var(--color-primario);
17     padding: var(--esp-md);
18     border-radius: var(--radio-borde);
19 }
```

Resultado: poder de las variables CSS

Si cambias `--color-primario: #e74c3c;` en `:root`, TODOS los elementos que usan `var(--color-primario)` cambian automaticamente: el boton, el header, el focus de los inputs, el color de los enlaces.

Prueba practica: abrir el archivo `variables.css`, cambiar el valor de `--color-primario` de `#0066cc` (azul) a `#e74c3c` (rojo), guardar, y observar cuantos elementos cambian en el navegador sin tocar ninguna otra linea de CSS.

Esto demuestra el principio DRY (Don't Repeat Yourself): cada valor se define una sola vez en un solo lugar. Si el cliente pide cambiar el color corporativo, se modifica una linea en lugar de buscar y reemplazar en decenas de archivos.

Las variables CSS tambien se pueden leer y modificar desde JavaScript con `document.documentElement.style.setProperty('--color-primario', '#e74c3c')`, lo que permite al usuario personalizar los colores del sitio en tiempo real.

Ejemplo 6 – Escala de espaciado consistente

```

1  :root {
2      /* Escala de 4px (cada valor es el doble del anterior) */
3      --esp-xs: 0.25rem; /* 4px */
4      --esp-sm: 0.5rem; /* 8px */
5      --esp-md: 1rem; /* 16px */
6      --esp-lg: 1.5rem; /* 24px */
7      --esp-xl: 2rem; /* 32px */
8  }
9
10 /* Usar consistentemente */
11 .campo-form { margin-bottom: var(--esp-md); }
12 .tarjeta { padding: var(--esp-lg); }
13 header { padding: var(--esp-md) var(--esp-lg); }

```

Donde se usa en la practica: variables.css – archivo completo

El archivo `variables.css` de la practica declara 22 variables organizadas en 5 categorias: colores principales y de superficies (6 variables de color como `-color-primario`, `-color-fondo`, `-color-texto`), tipografia (4 variables con `clamp()` para tamanos fluidos y `-fuente-base` con la pila de fuentes del sistema), espaciado (6 variables en escala de 4px: `-esp-xs` hasta `-esp-2xl`), layout (4 variables como `-ancho-max` y `-sombra-sm`) y transiciones (2 variables para duraciones estandar). El archivo `main.css` las consume con `var()` en cada propiedad, de modo que ningun valor numerico aparece repetido fuera de `:root`.

Nivel 4: Box Model – margin, padding, border**Concepto: El modelo de caja**

Todo elemento HTML es una caja con 4 capas, de adentro hacia afuera:

1. **Content:** el contenido real del elemento (texto, imagen, video). Su tamaño se controla con `width` y `height`.
2. **Padding:** espacio transparente entre el contenido y el borde. Empuja el contenido hacia adentro. Se define con `padding` (1-4 valores en orden del reloj: top, right, bottom, left).
3. **Border:** la linea visible alrededor del elemento. Se define con `border: ancho estilo color` (ej: `border: 2px solid #dee2e6`). `border-radius` redondea las esquinas.
4. **Margin:** espacio transparente entre este elemento y los elementos vecinos. Empuja otros elementos hacia afuera. `margin: 0 auto` centra horizontalmente. Los márgenes verticales entre elementos adyacentes colapsan: si un elemento tiene `margin-bottom: 20px` y el siguiente tiene `margin-top: 30px`, el espacio real entre ellos es 30px (el mayor), no 50px.

Propiedad adicional: `box-shadow` agrega una sombra visual que no afecta el flujo del documento (no ocupa espacio). Su sintaxis es: `box-shadow: desplazamiento-X desplazamiento-Y desenfoque color;`. Se puede agregar `inset` para sombras internas.

Ejemplo 7 – Padding, margin y border

```

1  .tarjeta {
2      /* Padding: espacio INTERIOR (entre el texto y el borde) */
3      padding: 1.5rem;
4      /* Border: linea visible alrededor */
5      border: 1px solid #dee2e6;
6      /* Border-radius: esquinas redondeadas */
7      border-radius: 12px;
8      /* Margin: espacio EXTERIOR (entre esta caja y las demas) */
9      margin-bottom: 1rem;
10     /* Box-shadow: sombra (no afecta el flujo del documento) */
11     box-shadow: 0 2px 8px rgba(0, 0, 0, 0.12);
12     /* Desglose: desplazamiento-X desplazamiento-Y desenfoque color
13         */
14 }

```

Ejemplo 8 – Shorthand de margin y padding (orden del reloj)

```

1  /* 4 valores: top right bottom left (sentido del reloj) */
2  padding: 10px 20px 10px 20px;
3
4  /* 2 valores: vertical horizontal */
5  /* arriba/abajo=10px, lados=20px */
6  padding: 10px 20px;
7
8  /* 1 valor: los 4 iguales */
9  padding: 1rem;
10
11 /* Centrar un bloque horizontalmente */
12 /* 0 arriba/abajo, auto a los lados */
13 margin: 0 auto;

```

Donde se usa en la practica: main.css – tarjetas y formulario

Las tarjetas usan `padding: var(-esp-lg)`, `border: 1px solid var(-color-borde)`, `border-radius: var(-radio-borde-lg)` y `box-shadow: var(-sombra-sm)`.

Nivel 5: Flexbox – distribuir en una dimension**Concepto: Flexbox**

Flexbox distribuye elementos en **una dimension**: una fila O una columna. Se activa con `display: flex` en el contenedor padre. Los hijos se distribuyen automaticamente.

Las propiedades mas importantes del contenedor flex son: `flex-direction` (fila o columna), `justify-content` (distribucion en el eje principal), `align-items` (alineacion en el eje secundario), `flex-wrap` (permitir que los hijos salten de linea) y `gap` (espacio entre hijos). Para los hijos, la propiedad `flex` combina tres valores: cuanto crece (`flex-grow`), cuanto se encoge (`flex-shrink`) y su tamano base (`flex-basis`).

Ejemplo 9 – Navbar horizontal con Flexbox

El header necesita dos grupos de contenido en los extremos opuestos: el titulo a la izquierda y la navegacion a la derecha. `space-between` los separa automaticamente. La lista de enlaces dentro del nav tambien es flex para distribuirlos horizontalmente con espacio uniforme.

```
1  header { /* Contenedor flex: distribuye hijos en fila */
2      display: flex;
3      align-items: center;          /* centrar verticalmente */
4      justify-content: space-between; /* titulo a la izq, nav a la der */
5      gap: 1rem;                  /* espacio entre hijos */
6      padding: 1rem 1.5rem;
7  }
8
9  nav ul { /* La lista de navegacion tambien es flex */
10     display: flex;
11     gap: 1rem;          /* espacio entre cada enlace */
12     list-style: none; /* quitar los puntos de la lista */
13 }
```

Ejemplo 10 – Tarjetas con flex-wrap (se ajustan al espacio)

Las tarjetas de datos del PFC deben adaptarse al espacio disponible: 3 por fila en desktop, 2 en tablet, 1 en mobile. Con `flex-wrap: wrap` y un `flex-basis` minimo, Flexbox lo resuelve sin media queries.

```
1  .tarjetas-grid {
2      display: flex;
3      flex-wrap: wrap; /* si no caben en una fila, saltan a la siguiente
4                      */
5      gap: 1rem;
6  }
7
8  .tarjeta {
9      flex: 1 1 260px;
10     /* Desglose:
11     flex-grow: 1    = CRECE para llenar el espacio disponible
12     flex-shrink: 1  = SE ENCOGE si el espacio es insuficiente
13     flex-basis: 260px = tamaño base minimo de 260px
14     Resultado: las tarjetas miden al menos 260px y crecen
15     para llenar la fila. Si no caben 2, se apilan. */
16 }
```

Ejemplo 11 – Formulario vertical con Flexbox

Cada campo del formulario tiene 3 elementos apilados verticalmente: el label arriba, el input en el medio y el mensaje de error abajo. `flex-direction: column` los organiza de arriba a abajo con un `gap` pequeno entre ellos.

```
1  .campo-form {
2      display: flex;
3      flex-direction: column; /* apilar label, input y error vertical */
4      gap: 0.25rem;          /* pequeno espacio entre cada uno */
5      margin-bottom: 1rem;
6  }
```

Donde se usa en la practica: main.css – header, tarjetas y formulario

La practica usa Flexbox en 3 lugares distintos, cada uno con una configuracion diferente:

- **Header:** `display: flex` con `justify-content: space-between` para poner el titulo a la izquierda y la navegacion a la derecha. `align-items: center` centra verticalmente ambos. `flex-wrap: wrap` permite que se apilen en pantallas pequenas.
- **Tarjetas:** `display: flex` con `flex-wrap: wrap` y `gap: 1rem`. Cada tarjeta tiene `flex: 1 1 260px`: crece para llenar el espacio, se encoge si falta, pero nunca mide menos de 260px. En pantalla ancha caben 3 tarjetas por fila; en mobile se apilan a 1.
- **Formulario:** cada `.campo-form` usa `flex-direction: column` para apilar verticalmente el label, el input y el mensaje de error, con `gap: 0.25rem` entre cada uno.

La diferencia clave con Grid es que Flexbox no controla las filas: si caben 3 tarjetas caben, si caben 2 se ajustan, si cabe 1 se apila. Grid forzaria exactamente 3 columnas.

Nivel 6: CSS Grid – layout de pagina completo**Concepto: CSS Grid**

Grid distribuye elementos en **dos dimensiones**: filas Y columnas al mismo tiempo. Se usa para el layout general de la pagina (header, main, aside, footer). Se activa con `display: grid` en el contenedor padre.

La propiedad `grid-template-areas` permite dibujar el layout visualmente en el codigo usando nombres de area entre comillas. Cada fila es un string y cada columna es un nombre separado por espacios. Luego, cada elemento HTML se asigna a su area con `grid-area: nombre;`. Esta sintaxis es la mas legible y mantenible para layouts de pagina.

Ejemplo 12 – Grid basico con areas nombradas

Usa `grid-template-areas` para nombrar cada zona del layout. Al cambiar las media queries, se reorganiza el layout modificando solo las areas, sin tocar el HTML.

```

1  .layout {
2      display: grid;
3      grid-template-areas:
4          "header header"      /* header ocupa las 2 columnas */
5          "main  aside"        /* main a la izquierda, aside a la derecha */
6          "footer footer";     /* footer ocupa las 2 columnas */
7      grid-template-columns: 1fr 280px; /* main=flexible, aside=280px */
8      gap: 1rem;
9      min-height: 100vh;      /* minimo el alto de la ventana */
10 }
11
12 /* Asignar cada elemento a su area */
13 header { grid-area: header; }
14 main    { grid-area: main; }
15 aside   { grid-area: aside; }
16 footer  { grid-area: footer; }

```

Concepto: fr (fraction unit)

1fr significa “una fraccion del espacio disponible”. En `grid-template-columns: 1fr 280px`, la primera columna toma todo el espacio que sobra despues de asignar 280px al aside. Si el contenedor mide 1000px: $\text{main} = 1000 - 280 - \text{gap} = 704\text{px}$.

Se pueden combinar multiples **fr**: `grid-template-columns: 2fr 1fr` crea dos columnas donde la primera es el doble de ancha que la segunda. La funcion `repeat()` simplifica la repeticion: `repeat(3, 1fr)` equivale a `1fr 1fr 1fr` (3 columnas iguales).

Donde se usa en la practica: main.css – layout principal

Este ejemplo es exactamente el layout de la practica en la version tablet/desktop.

Nivel 7: Responsive mobile-first con media queries**Concepto: Mobile-first**

Se escribe primero el CSS para la pantalla mas pequena (mobile). Luego se agregan media queries con `min-width` para pantallas mas grandes. Esto es mas eficiente porque el CSS base es el mas simple y las media queries solo agregan complejidad cuando hay espacio.

Por que mobile-first y no desktop-first:

- En el enfoque **desktop-first**, el CSS base es complejo (2 columnas, margenes amplios) y las media queries con `max-width` van quitando cosas para pantallas pequenas. Esto genera mas codigo y es mas propenso a errores.
- En el enfoque **mobile-first**, el CSS base es simple (1 columna, sin margenes laterales) y las media queries con `min-width` agregan complejidad solo cuando hay espacio para ello. Menos codigo, menos bugs.
- Google indexa primero la version mobile del sitio (**mobile-first indexing**), asi que el CSS base debe ser el mobile.

Breakpoints mas usados: 768px (tablet), 1024px (desktop), 1200px (pantalla ancha). En la practica usamos los primeros dos.

Ejemplo 13a – Mobile (base, sin media query)

```
1  /* MOBILE: 1 columna (CSS base, sin media query) */
2  .layout {
3      display: grid;
4      grid-template-areas:
5          "header"
6          "main"
7          "aside"
8          "footer";
9      gap: 1rem;
10     padding: 1rem;
11 }
```

Ejemplo 13b – Tablet y Desktop (media queries)

```
1  /* TABLET (768px+): 2 columnas */
2  @media (min-width: 768px) {
3    .layout {
4      grid-template-areas:
5        "header header"
6        "main  aside"
7        "footer footer";
8      grid-template-columns: 1fr 280px;
9      padding: 1.5rem;
10   }
11 }
12
13 /* DESKTOP (1024px+): centrado */
14 @media (min-width: 1024px) {
15   .layout {
16     grid-template-columns: 1fr 320px;
17     max-width: 1200px;
18     margin: 0 auto;
19     padding: 2rem;
20   }
21 }
```

Como verificar el responsive

F12 en Chrome → icono de dispositivo movil (Toggle device toolbar) → cambiar el ancho: 375px (mobile, 1 columna) → 768px (tablet, 2 columnas) → 1024px (desktop, centrado). El aside pasa de estar debajo del main a estar al lado.

Donde se usa en la practica: main.css – el layout completo

Este ejemplo es exactamente el layout responsive de la practica con las mismas 3 etapas y los mismos breakpoints.

Nivel 8: Estilos de formulario accesibles**Ejemplo 14a – Base de todos los inputs**

```
1  input, select, textarea {
2    padding: 0.5rem 1rem;
3    border: 2px solid #dee2e6;
4    border-radius: 6px;
5    font-size: 1rem;
6    font-family: inherit; /* hereda la fuente del body */
7    background: var(--color-fondo);
8    color: var(--color-texto);
9    transition: border-color 150ms ease;
10 }
```

Ejemplo 14b – Focus, validacion y mensajes de error

```

1  /* Focus: anillo azul al hacer clic o Tab */
2  input:focus, select:focus, textarea:focus {
3      outline: 3px solid #0066cc;
4      outline-offset: 2px;
5      border-color: #0066cc;
6  }
7  /* Invalido SOLO si el usuario ya escribio algo */
8  input:invalid:not(:placeholder-shown) {
9      border-color: #dc3545; /* borde rojo */
10 }
11 /* Textos de ayuda y error */
12 .mensaje-error { color: #dc3545; font-size: 0.875rem; }
13 .texto-ayuda { color: #555; font-size: 0.875rem; }

```

Concepto: :focus-visible vs :focus

:focus se activa siempre que el elemento recibe foco (teclado O raton). :focus-visible se activa solo cuando el foco es por teclado (Tab). Los botones usan :focus-visible para no mostrar el outline cuando el usuario hace clic con el raton (que es molesto) pero si cuando navega con Tab (que es necesario para accesibilidad).

Los inputs de formulario usan :focus (no :focus-visible) porque el outline debe aparecer tanto al hacer clic como al navegar con Tab, ya que el usuario necesita ver cual campo esta editando en ambos casos. Las mejores practicas de accesibilidad exigen que el outline sea de al menos 3px y tenga un outline-offset de 2-3px para que no se confunda con el borde del elemento.

Donde se usa en la practica: main.css – formulario

La practica usa exactamente estos estilos. El selector :invalid:not(:placeholder-shown) evita que el campo se muestre rojo antes de que el usuario escriba (porque un campo vacio es invalido si tiene required, pero no queremos asustarlo antes de que empiece).

Nivel 9: Botones con transiciones y estados**Ejemplo 15a – Boton: estilo base**

```

1  .btn-primario {
2      background: var(--color-primario);
3      color: #fff;
4      padding: 0.5rem 2rem;
5      border: none;
6      border-radius: 6px;
7      font: 1rem inherit;
8      cursor: pointer;
9      transition: background 150ms ease, transform 150ms ease;
10 }

```

Ejemplo 15b – Boton: hover, active, focus-visible y disabled

```

1  /* Hover: cuando el ratón pasa por encima del botón. Se oscurece el
2     fondo para dar feedback visual al usuario acerca del boton.*/
3  .btn-primario:hover {
4     background: var(--color-primario-hover);
5  }
6  /* Active: Al mantener presionado el botón con el clic. scale(0.98)
7     lo encoge un 2% simulando que se "hunde", como un boton fisico. */
8  .btn-primario:active {
9     transform: scale(0.98);
10  }
11  /* Focus-visible: aparece SOLO cuando llega al boton con la tecla Tab,
12     NO con clic. El outline de 3px + offset de 3px crea un anillo visible
13     separado del borde del boton. Es obligatorio para accesibilidad
14     (WCAG 2.1). */
15  .btn-primario:focus-visible {
16     outline: 3px solid var(--color-primario);
17     outline-offset: 3px;
18  }
19  /* Disabled: botón deshabilitado (ej: formulario incompleto o
20     esperando respuesta del servidor). opacity 0.5 lo hace
21     semitransparente. cursor not-allowed muestra icono de prohibido.
22     pointer-events none ignora todos los clics. */
23  .btn-primario:disabled {
24     opacity: 0.5;
25     cursor: not-allowed;
26     pointer-events: none;
27  }

```

Concepto: transition

La propiedad **transition** anima los cambios de valor de otras atributos CSS. Su sintaxis es:

transition: propiedad duracion curva retardo;

- **propiedad:** cual propiedad animar (background, color, transform, all).
- **duracion:** cuanto dura la animacion (150ms, 0.3s). Regla: 100-300ms para hover de botones/enlaces, 300-500ms para cambios de layout o tema.
- **curva:** la aceleracion de la animacion. **ease** (lento-rapido-lento, la mas natural), **linear** (velocidad constante), **ease-in** (empieza lento), **ease-out** (termina lento), **ease-in-out** (lento en ambos extremos).
- **retardo** (opcional): esperar antes de iniciar (50ms, 0.1s).

Se pueden animar multiples propiedades separandolas con coma: **transition: background 150ms ease, transform 150ms ease;**. Usar **all** anima todas las propiedades que cambien, pero es menos eficiente porque el navegador debe vigilar todas.

Propiedades que se pueden animar: las que tienen valores intermedios calculables: colores (color, background), tamanos (width, height, padding), posiciones (top, left), transformaciones (transform), opacidad (opacity). **No se pueden animar:** display, font-family, position.

Nivel 10: Modo oscuro automatico y manual

Tip: el modo oscuro automatico no requiere JavaScript. CSS detecta la preferencia del sistema operativo con la media query `prefers-color-scheme: dark`. Solo se redefinen las variables de color en `:root`; el resto del CSS no cambia ni una linea porque todo usa `var()`. Esta es la ventaja principal de haber definido variables en el Nivel 3: un solo punto de cambio controla toda la apariencia del sitio.

Ejemplo 16 – Modo oscuro con media query del sistema

```

1  /* === MODO CLARO (base, siempre se carga primero) === */
2  :root {
3      --color-fondo:          #ffffff; /* blanco */
4      --color-fondo-tarjeta: #f8f9fa; /* gris muy claro */
5      --color-texto:          #1a1a1a; /* casi negro */
6      --color-texto-suave:    #555555; /* gris medio */
7      --color-borde:          #dee2e6; /* gris claro */
8  }
9  /* === MODO OSCURO AUTOMATICO ===
10 Esta media query detecta si el sistema operativo del usuario
11 tiene configurado el tema oscuro. Solo redefine las variables
12 de color; el resto del CSS no cambia ni una linea. */
13 @media (prefers-color-scheme: dark) {
14     :root {
15         --color-fondo:          #0f172a; /* azul muy oscuro */
16         --color-fondo-tarjeta: #1e293b; /* azul oscuro */
17         --color-texto:          #f1f5f9; /* casi blanco */
18         --color-texto-suave:    #94a3b8; /* gris azulado */
19         --color-borde:          #334155; /* gris oscuro */
20     }
21 }

```

Como probar el modo oscuro automatico sin cambiar tu SO

En Chrome: F12 → Ctrl+Shift+P → escribir “rendering” → Show Rendering → buscar “Emulate CSS media feature prefers-color-scheme” → seleccionar **dark**. El sitio cambia al instante. Para volver: seleccionar **light** o **No emulation**.

En Firefox: F12 → Inspector → en la barra superior hay un icono de sol/luna que alterna entre claro y oscuro.

Errores frecuentes si no funciona: (1) el body usa un color fijo como `background: #fff` en lugar de `var(--color-fondo)`; (2) la media query tiene un error de sintaxis (verificar que diga exactamente `prefers-color-scheme: dark` sin mayusculas); (3) hay un `!important` en otro lugar que sobrescribe la variable.

Tip: el modo manual complementa al automatico. Si el usuario quiere modo claro aunque su SO este en oscuro (o viceversa), el boton de tema agrega el atributo `data-tema` al `<html>` via JavaScript, y el selector CSS `[data-tema=."oscuro"]` tiene prioridad sobre la media query. La preferencia se guarda en `localStorage` para que persista al recargar la pagina o cerrar el navegador. Sin `localStorage`, el usuario tendria que presionar el boton cada vez que entra al sitio.

Ejemplo 17 – Modo oscuro manual con boton (atributo data-tema)

```

1  /* === MODO OSCURO MANUAL ===
2  Cuando el usuario presiona el boton de tema, JavaScript ejecuta:
3  document.documentElement.setAttribute('data-tema', 'oscuro');
4  Esto agrega data-tema="oscuro" al <html> */
5  [data-tema="oscuro"] {
6      --color-fondo:          #0f172a;
7      --color-fondo-tarjeta: #1e293b;
8      --color-texto:          #f1f5f9;
9      --color-texto-suave:    #94a3b8;
10     --color-borde:          #334155;
11 }
12 /* === COMBINACION: auto + manual ===
13 Si el SO tiene modo oscuro, activar el tema oscuro EXCEPTO cuando
14 el usuario eligio explicitamente el modo claro con el boton.
15 :not([data-tema="claro"]) excluye ese caso. */
16 @media (prefers-color-scheme: dark) {
17     :root:not([data-tema="claro"]) {
18         --color-fondo:          #0f172a;
19         --color-fondo-tarjeta: #1e293b;
20         --color-texto:          #f1f5f9;
21         --color-texto-suave:    #94a3b8;
22         --color-borde:          #334155;
23     }
24 }

```

Prioridad: manual gana sobre automatico

El truco esta en el selector `:root:not([data-tema="claro"])`. Si el SO tiene modo oscuro activo pero el usuario presiono el boton para elegir modo claro, JavaScript pone `data-tema="claro"` en el `<html>`. El `:not()` excluye ese caso y el modo claro prevalece. Sin el `:not()`, el modo oscuro del SO siempre ganaria sobre la eleccion del usuario.

Flujo completo de la logica:

1. El usuario entra por primera vez → no hay `data-tema` → se aplica la preferencia del SO via media query.
2. El usuario presiona el boton de tema → JavaScript agrega `data-tema="oscuro"` o `claro"` al `<html>` y lo guarda en `localStorage`.
3. El usuario vuelve al sitio → JavaScript lee `localStorage` y restaura el `data-tema` guardado, ignorando la preferencia del SO.
4. Si el usuario nunca toco el boton, siempre respeta su SO automaticamente.

Ejemplo 18 – Transicion suave entre modos

```

1  body {
2      background-color: var(--color-fondo);
3      color: var(--color-texto);
4      transition: background-color 300ms ease, color 300ms ease;
5  }/* Transicion: el cambio de color es gradual, no brusco */

```

Donde se usa en la practica: variables.css – modo oscuro completo

La practica combina las tres tecnicas: variables en `:root` (claro), media query para auto-deteccion del SO, y selector `[data-tema=.°scuro]` para el boton. La transicion en el body hace que el cambio sea suave.

Nivel 11: Animaciones y accesibilidad

Tip: las animaciones CSS se definen en dos partes: primero el `@keyframes` que describe los estados inicial (**from**) y final (**to**) de la animacion, y luego la propiedad `animation` que conecta el keyframe con el elemento y define duracion, curva y comportamiento. El valor `both` en `animation-fill-mode` hace que el elemento mantenga el estado final al terminar y aplique el estado inicial antes de empezar.

Ejemplo 19a – Definir la animacion @keyframes

```

1  @keyframes aparecer {
2      from { opacity: 0;                /* invisible */
3          transform: translateY(12px); /* 12px abajo */
4      }
5      to {
6          opacity: 1;                /* visible */
7          transform: translateY(0);   /* posicion normal */
8      }
9  }
```

Tip: en la practica, cada tarjeta tiene un `animation-delay` diferente calculado con JavaScript: `style="animation-delay: ${i * 60}ms"`, donde `i` es el indice de la tarjeta. Esto crea un efecto escalonado donde las tarjetas aparecen una tras otra en lugar de todas al mismo tiempo.

Ejemplo 19b – Aplicar y desactivar por accesibilidad

```

1  .tarjeta { /* nombre duracion curva fill-mode */
2      animation: aparecer 0.4s ease both;
3  }
4  /* OBLIGATORIO: desactivar para usuarios con sensibilidad */
5  @media (prefers-reduced-motion: reduce) {
6      .tarjeta { animation: none; }
7  }
```

Tip: Lighthouse verifica automaticamente si el sitio respeta `prefers-reduced-motion`. Si hay animaciones activas sin esta media query, el score de Accessibility baja. La regla es simple: toda propiedad `animation` o `transition` debe tener su contraparte `@media (prefers-reduced-motion: reduce)` que la desactive con `animation: none` o `transition: none`.

Concepto: prefers-reduced-motion

Hay usuarios que experimentan mareo, nauseas o convulsiones con animaciones. Sus sistemas operativos tienen una opcion para “Reducir movimiento”. La media query `prefers-reduced-motion: reduce` detecta esa preferencia. Es **obligatorio** desactivar todas las animaciones cuando esta activa. Lighthouse lo verifica.

11.1 Enlace de salto (skip link) accesible

Ejemplo 20 – Skip link oculto que aparece con Tab

```

1  /* Oculto por defecto (fuera de la pantalla) */
2  .saltar-nav {
3      position: absolute;
4      top: -100%;          /* fuera de la vista */
5      left: 0;
6      padding: 0.5rem 1rem;
7      background: #0066cc;
8      color: #fff;
9      text-decoration: none;
10     z-index: 9999;        /* siempre encima de todo */
11     transition: top 150ms ease;
12 }
13
14 /* Aparece cuando el usuario presiona Tab */
15 .saltar-nav:focus {
16     top: 0; /* entra a la pantalla */
17 }

```

Donde se usa en la practica: main.css – accesibilidad

El skip link es el primer elemento del <body> en el HTML: Ir al contenido principal. Es invisible hasta que el usuario presiona Tab, momento en que aparece en la esquina superior izquierda.

Nivel FINAL: Mapa completo del CSS de la practica

Verificacion

Si entendiste los 20 ejemplos anteriores, puedes leer y escribir cada linea del Paso 3 de la practica. La Tabla 1 mapea cada concepto a su ubicacion exacta en el codigo de la practica y al ejemplo correspondiente en este documento.

Cuadro 1: Mapa completo: concepto CSS → ubicacion en la practica → ejemplo en este documento.

#	Concepto CSS	Donde se usa en la practica	Ejemplo en este documento
1	Reset con box-sizing	Primeras lineas de main.css	Ejemplo 3
2	Unidades rem, em, %, vw, clamp()	Variables de tipografia en variables.css	Ejemplo 4
3	Variables CSS en :root	Archivo variables.css completo	Ejemplos 5 y 6
4	Padding, border, border-radius, box-shadow	Tarjetas, formulario, header	Ejemplos 7 y 8
5	Flexbox horizontal	Header: titulo + nav con space-between	Ejemplo 9

#	Concepto CSS	Donde se usa en la practica	Ejemplo en este documento
6	Flexbox con flex-wrap	Tarjetas: flex: 1 1 260px	Ejemplo 10
7	Flexbox vertical	Campos del formulario: flex-direction: column	Ejemplo 11
8	CSS Grid con areas nombradas	Layout principal: header, main, aside, footer	Ejemplo 12
9	Media queries mobile-first	3 breakpoints: base, 768px, 1024px	Ejemplo 13a y 13b
10	Focus, :invalid, :focus-visible	Inputs con outline y validacion visual	Ejemplo 14a y 14b
11	Boton con 5 estados	.btn-primario: normal, hover, active, focus, disabled	Ejemplo 15a y 15b
12	Modo oscuro automatico	@media (prefers-color-scheme: dark) en variables.css	Ejemplo 16
13	Modo oscuro manual	[data-tema="oscuro"] + :not() en variables.css	Ejemplo 17
14	Transicion suave de tema	body: transition background-color 300ms en main.css	Ejemplo 18
15	@keyframes y animation	Animacion de entrada de tarjetas en main.css	Ejemplo 19a
16	prefers-reduced-motion	Desactivar animaciones para accesibilidad	Ejemplo 19b
17	Skip link accesible	Enlace oculto que aparece con Tab en main.css	Ejemplo 20

Estas listo

Si puedes explicar cada fila de esta tabla, puedes implementar el Paso 3 completo de la practica. Abre la guia de la PE U1, ve al Paso 3, y veras que cada bloque de CSS corresponde a uno de estos 20 ejemplos.